

Desempenho de Funções de Criptografia Integrada em Arquiteturas de Armazenamento Não-Relacionais: Um Estudo Comparativo

Autoria Omitida para Revisão às Cegas

¹Filiação Omitida para Revisão às Cegas

Abstract. *The adoption of NoSQL databases in data-intensive applications raises questions about the cost of cryptographic functions at the persistence layer. This paper evaluates the throughput, latency, and resource consumption of AES-256, SHA-256, HMAC, Bcrypt, and Argon2 integrated into MongoDB, Redis, and Cassandra. Tested on dedicated hardware, results show Redis maximizes throughput for lightweight cryptography (up to 39,757 ops/s), while MongoDB provides balanced persistence. Password-hashing functions (Bcrypt/Argon2) proved strictly CPU-bound (~20-30 ops/s), rendering the database choice irrelevant. Cassandra showed severe degradation for large payloads. This study guides architecture choices for secure environments.*

Resumo. *A adoção de bancos NoSQL em aplicações com dados sensíveis impõe desafios sobre o custo de funções criptográficas na camada de persistência. Este artigo avalia o throughput, latência e consumo de recursos do AES-256, SHA-256, HMAC, Bcrypt e Argon2 integrados ao MongoDB, Redis e Cassandra. Ensaios em hardware dedicado revelam que o Redis maximiza o rendimento em cifras leves (até 39.757 ops/s), enquanto o MongoDB entrega resiliência equilibrada. Algoritmos complexos de senhas provaram-se estritamente CPU-bound (~20-30 ops/s), independentemente do banco. O Cassandra sofreu severa degradação com grandes payloads. Este estudo baliza decisões arquiteturais em segurança.*

1. Introdução

A garantia do sigilo informacional consolidou-se como um pilar inflexível no desenho de ambientes computacionais contemporâneos. Sob a égide de legislações como a Lei Geral de Proteção de Dados (LGPD), as organizações encontram-se obrigadas a adotar mecanismos que blindem registros sensíveis [Shastri et al. 2019]. A aplicação de algoritmos criptográficos diretamente na camada de persistência ergue-se como a barreira de defesa mais premente [Rosa and Lazarin 2023]. Todavia, de forma distinta dos ecossistemas relacionais tradicionais, as arquiteturas não-relacionais (NoSQL) foram projetadas primariamente para alta escalabilidade, raramente oferecendo a encriptação nativa isenta de custos operacionais. O ato contínuo de cifrar e decifrar exige um esforço substancial do maquinário, comprometendo as premissas originais de alto *throughput* que motivaram a adoção dessas plataformas.

Para fazer frente à explosão volumétrica moderna, sistemas como MongoDB (orientado a documentos), Redis (*in-memory*) e Apache Cassandra (colunas amplas) tornaram-se indispensáveis. Contudo, a reação de cada modelo à imposição da segurança

é heterogênea. A ausência de parâmetros quantitativos sobre essa degradação na camada de aplicação dificulta a tomada de decisão estrutural por engenheiros e pesquisadores de cibersegurança.

Este trabalho preenche essa lacuna ao propor e executar um *benchmark* padronizado focado em mensurar o impacto direto de operações integradas de persistência e criptografia. O objetivo é desvendar como o *overhead* protetivo oscila em função do algoritmo adotado e do tamanho do registro, gerando um artefato empírico de suporte à decisão. O artigo organiza-se explorando a fundamentação teórica (Seção 2), os trabalhos correlatos (Seção 3), a metodologia de ensaio (Seção 4), a análise das métricas (Seção 5) e as considerações finais (Seção 6).

2. Fundamentação Teórica

Nesta seção, estabelecem-se as bases conceituais que norteiam a pesquisa. Aborda-se a necessidade intrínseca de governança de dados e, em seguida, detalham-se os modelos de armazenamento e os algoritmos criptográficos avaliados no escopo do estudo.

2.1. Governança e Modelos Flexíveis

A governança da privacidade apoia-se em legislações que exigem proteção desde a concepção fundacional. Para atender a essas demandas, o mercado adota bancos NoSQL. A seleção das três plataformas avaliadas neste estudo justifica-se por cobrirem os principais e mais distintos paradigmas arquiteturais do ecossistema, permitindo observar como diferentes engenharias reagem à mesma carga criptográfica: o MongoDB lidera a adoção no modelo orientado a documentos (estruturas flexíveis); o Redis representa o armazenamento chave-valor operante de forma transitória e primária na memória (latência de microssegundos); e o Cassandra exemplifica o paradigma de colunas amplas (distribuição otimizada para tolerância a falhas em volumes massivos de escrita).

2.2. Algoritmos Criptográficos

O estudo examina o *Advanced Encryption Standard* (AES-256) operando no modo *Cipher Feedback* (CFB) para ofuscação de blocos. Para a garantia de integridade, avaliam-se as funções de *hash* SHA-256 e os códigos de autenticação HMAC-SHA-256 [Stallings 2015]. Ademais, investigam-se funções exaustivas de derivação de senhas com fator de esforço configurável: o Bcrypt (baseado no cifrador Blowfish) e o Argon2 (otimizado contra ataques de força bruta paralelos em GPUs) [Biryukov et al. 2016].

3. Trabalhos Relacionados

A investigação científica tem dedicado esforços contínuos para precificar o custo das barreiras de segurança. Nas estruturas relacionais, Rosa e Lazarin (2023) identificaram penalidades de até 16,2% no PostgreSQL sob regimes rígidos de proteção encriptada [Rosa and Lazarin 2023]. No ecossistema flexível contemporâneo, pontuou-se analiticamente que a maleabilidade dos modelos NoSQL ostenta a capacidade teórica de mitigar parte dessas perdas de *throughput* [Kilonzo 2020].

Avaliando o rendimento bruto das arquiteturas isentas de ofuscação matemática, Nasar et al. (2022) consolidaram a primazia do Redis em velocidade primordial via YCSB

[Nasar et al. 2022]. Complementarmente, Saran e Saran (2019) constataram que o MongoDB processou um volume de dados cerca de 45% superior ao Cassandra sob testes genéricos de segurança [Saran and Saran 2019]. O presente estudo avança materialmente sobre essas iniciativas ao instrumentar, de forma direta e unificada, rotinas criptográficas modernas sobre conjuntos de dados realistas para aferir o *overhead* sistêmico integrado.

4. Metodologia

O arcabouço experimental foi desenhado para isolar matematicamente o custo da segurança na camada de arquivamento. A seguir, descrevem-se a infraestrutura, a composição da massa informacional, os protocolos de execução padronizados e as garantias de auditoria do estudo.

4.1. Infraestrutura e Massa Informacional

Os ensaios transcorreram em maquinário dedicado (Processador de 6 núcleos físicos, 32 GB de memória de acesso imediato e sistema operativo Debian GNU/Linux 12). Para neutralizar a concorrência por recursos mecânicos de gravação, cada matriz foi instalada em um disco rígido físico exclusivo. Utilizou-se um conjunto de testes padrão universitário (YCSB) [Cooper et al. 2010] para estabelecer o referencial limpo de operações, em conjunto com o acervo autêntico RT-IoT2022 [UCI Machine Learning Repository 2022], contendo atributos sensíveis de telemetria, para compor os ensaios protegidos.

4.2. Protocolo Experimental

Foram desenhadas avaliações cobrindo as interações fundamentais de manipulação informacional intimamente integradas às fórmulas matemáticas AES-256, SHA-256, HMAC, Bcrypt e Argon2. Para mensurar a influência da densidade do documento, testaram-se quatro dimensões: 64, 256, 1.024 e 4.096 bytes. Realizaram-se 10.000 iterações por dimensão, com amostragem concorrente de utilização da capacidade de raciocínio lógico e retenção de memória coletada rigorosamente a cada 50 milissegundos.

4.3. Transparência e Reprodutibilidade

A reprodutibilidade é essencial para a validação de qualquer ensaio comparativo. Para assegurar a transparência do método e permitir que a comunidade acadêmica audite, replique ou expanda os cenários aqui documentados, todos os materiais de teste foram disponibilizados de forma aberta. Os roteiros de execução, os parâmetros de configuração do ambiente e os resultados brutos encontram-se publicamente acessíveis através do endereço: <https://caiosenaf15.github.io/benchmark-NoSQL/>.

5. Resultados e Discussão

Os ensaios empíricos evidenciaram disparidades operacionais severas ancoradas nas distintas engenharias subjacentes de cada plataforma. Esta seção dissecou o comportamento da cadência de *throughput*, o impacto de algoritmos laboriosos e a consequente absorção de recursos sistêmicos.

Tabela 1. Throughput de inserção com AES-256 por tamanho de payload (ops/s)

Modelo	64 Bytes	256 Bytes	1.024 Bytes	4.096 Bytes
Redis	19.913,9	20.150,5	19.313,3	18.118,7
MongoDB	9.256,1	6.624,9	8.760,0	8.039,6
Cassandra	4.239,5	5.603,0	4.898,1	22,7

5.1. Análise do Throughput e Impacto do Payload

A Tabela 1 detalha o *throughput* registrado durante a operação de inserção combinada à encriptação simétrica de fluxo.

A arquitetura *in-memory* do Redis supre sumariamente os gargalos de I/O mecânico, permitindo o escoamento máximo do AES-256 com uma degradação marginal de apenas 9% entre o menor e o maior *payload*. O MongoDB apresentou uma resiliência equilibrada e constante. O Cassandra, todavia, sofreu um colapso operacional catastrófico ao atingir os *payloads* mais densos (22,7 ops/s). Esse fenômeno atrela-se à saturação aguda dos mecanismos de *compaction* e ao acúmulo de *tombstones* lógicos em instâncias que não operam de forma distribuída (*single-node*).

5.2. Independência em Algoritmos CPU-Bound

A Tabela 2 sintetiza o *throughput* médio absoluto das plataformas ao enfrentar diferentes níveis de complexidade matemática.

Tabela 2. Throughput Médio Comparativo por Operação e Algoritmo (ops/s)

Cenário Protegido	MongoDB	Redis	Cassandra
INSERT + SHA-256	11.568,8	35.126,2	3.349,2
INSERT + AES-256	8.170,1	19.374,1	3.690,8
READ + Decrypt (AES)	9.844,5	21.125,3	3.898,3
INSERT + Argon2	30,9	29,6	32,7
INSERT + Bcrypt	20,2	20,3	19,5

Sob a aplicação intencional de lógicas laboriosas de derivação de senhas (Argon2 e Bcrypt), o *throughput* estabilizou-se unissonamente na faixa de 20 a 30 ops/s em todas as fundações. Esse comportamento comprova que o gargalo computacional desloca-se integralmente para os núcleos do processador, caracterizando a operação como estritamente *CPU-bound*. Nesses casos, a escolha da arquitetura de persistência torna-se indiferente para o atraso sistêmico.

5.3. Consumo de Recursos (CPU e RAM)

O monitoramento do uso do processador em operações leves (AES, SHA) apresentou oscilações brandas, contidas entre 7% e 17%. Contudo, a métrica de retenção de RAM desvelou abismos arquiteturais: o Redis demonstrou eficiência extrema (apropriando-se de apenas 2,3% a 2,7% do sistema), enquanto o MongoDB exigiu uma alocação marginalmente superior (3,0% a 3,8%). Em drástico contraste, o Cassandra confiscou constantemente mais de 30% da memória disponível, refletindo o pesado *footprint* exigido pela Máquina Virtual Java (JVM) em tempo de execução.

6. Conclusão

Este estudo forneceu um *benchmark* empírico quantificando o verdadeiro custo de alinhar segurança criptográfica rigorosa à persistência NoSQL. Constatou-se que o Redis isolou a barreira mecânica atingindo picos de fluidez formidáveis; que o MongoDB comprovou-se como o terreno de persistência estruturada mais versátil; e atestou-se a limitação temporal unânime imposta por algoritmos de exaustão, independentemente do SGBD escolhido. Tais artefatos são fundamentais para que a comunidade acadêmica e de segurança arquitecte defesas sistêmicas balizadas por dados robustos. Futuras expansões devem englobar avaliações em ecossistemas distribuídos (*clusters*) e cargas transacionais multithread concorrentes.

Referências

- Biryukov, A., Dinu, D., and Khovratovich, D. (2016). Argon2: New generation of memory-hard functions for password hashing and other applications. In *IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 292–302. IEEE.
- Cooper, B. F., Silberstein, A., Tam, E., Ramakrishnan, R., and Sears, R. (2010). Benchmarking cloud serving systems with ycsb. In *Proceedings of the 1st ACM symposium on Cloud computing*, pages 143–154.
- Kilonzo, A. M. (2020). Impact of symmetric-key encryption on performance of relational and nosql database to preserve data confidentiality. Master’s thesis, United States International University-Africa.
- Nasar, M. et al. (2022). A study of performance and comparison of nosql databases: Mongodb, cassandra and redis using ycsb. *Indian Journal of Science and Technology*, 15(29):1–10.
- Rosa, A. S. and Lazarin, N. M. (2023). Uma análise de desempenho de funções de encriptação nativas de sgbds open source. In *Anais do 23º Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais (SBSeg)*. SBC.
- Saran, M. and Saran, N. (2019). A comparative security analysis of cassandra and mongodb nosql databases. *Uluslararası Bilgi Güvenliği Mühendisliği Dergisi*, 5(2):1–11.
- Shastri, S. et al. (2019). Understanding and benchmarking the impact of gdpr on database systems. *arXiv preprint arXiv:1910.00728*.
- Stallings, W. (2015). *Criptografia e Segurança de Redes: Princípios e Práticas*. Pearson Education do Brasil, 6 edition.
- UCI Machine Learning Repository (2022). Rt-iot2022 dataset. id 942. Disponível em: <https://archive.ics.uci.edu/dataset/942/rt-iot2022>.